

MEMORIA TÉCNICA

Autor: Hugo García Ruiz

Asignatura: Programación

Fecha: 15 Mayo 2026



1. DESCRIPCIÓN GENERAL DEL PROYECTO

1.1. Contexto y objetivos del proyecto

El presente proyecto consiste en el desarrollo de una aplicación en Java para la gestión integral de una liga de e-sports. La aplicación está diseñada para ser ejecutada por consola mediante un sistema de menús interactivos que permiten al usuario administrar todos los aspectos de la competición, desde el registro de personas, que incluye jugadores y entrenadores, hasta la generación del calendario, la disputa de partidos, la gestión de incidencias, las apuestas y la visualización de estadísticas. Se trata de un sistema completo que refleja el dominio de los contenidos trabajados durante el curso, incluyendo programación estructurada, programación modular, programación orientada a objetos, arrays, matrices, excepciones, clases abstractas, interfaces y estructuras dinámicas.

1.2. Funcionalidades obligatorias implementadas

Entre los objetivos alcanzados se encuentra la gestión completa de personas de la liga, permitiendo dar de alta, listar, buscar por identificador, modificar datos y eliminar personas cuando procede. También se ha implementado la gestión de equipos, incluyendo su creación, eliminación y consulta de costes. En cuanto a las plantillas, cada equipo dispone de un array fijo de cinco jugadores titulares y una lista dinámica de jugadores suplentes, con funcionalidades para añadir suplentes, eliminarlos, recorrer la colección y promover suplentes a titulares cuando procede, respetando siempre la regla de que no puede haber dos titulares con el mismo rol dentro del mismo equipo.

1.3. Sistema de calendario y partidos

El sistema incluye un calendario implementado mediante una matriz bidimensional donde cada celda almacena los puntos obtenidos por cada equipo en cada jornada. Los partidos pendientes se almacenan en una cola con comportamiento FIFO, permitiendo encolar partidos, mostrar el siguiente partido pendiente, disputarlo y mostrar todos los partidos pendientes. Además, se dispone de una pila con comportamiento LIFO para almacenar las últimas acciones realizadas por el usuario, lo que permite registrar acciones, mostrar la última acción realizada, mostrar el historial completo y deshacer operaciones de forma parcial.

1.4. Registro de partidos, incidencias y convocatorias

El registro de partidos contempla la creación de partidos, el registro de resultados, el cálculo del ganador, la actualización de estadísticas de equipos y jugadores, y la prevención de registrar dos veces un mismo partido o de crear partidos no válidos como un equipo contra sí mismo. En cuanto a incidencias y sanciones, se ha creado una clase Incidencia que permite registrar incidencias de diversos tipos, listarlas, buscarlas por equipo o jugador, aplicar sanciones y evitar que un jugador sancionado entre en una convocatoria válida. Para disputar un partido, cada equipo debe generar una convocatoria válida que cumpla con las reglas de participar

exactamente cinco titulares, no incluir jugadores sancionados y no repetir roles, pudiendo incorporar un suplente compatible si falta un titular.

1.5. Estadísticas, clasificación y funcionalidad extra

Finalmente, el programa genera y muestra estadísticas de jugadores como partidas jugadas, MVP obtenidos, rendimiento medio, estado de sanción y coste, así como estadísticas de equipos como victorias, derrotas, puntos a favor y en contra, diferencia de puntos y coste total de plantilla. La clasificación general se muestra ordenada por número de victorias, diferencia de puntos y nombre del equipo, calculándose a partir de los datos almacenados sin introducción manual. Como funcionalidad extra, se ha implementado un sistema de apuestas que permite a los usuarios apostar por equipos en partidos pendientes, con cuotas calculadas dinámicamente en función de las victorias y derrotas del equipo, resolviéndose automáticamente al disputarse el partido y actualizando el saldo del administrador.

1.6. Estructura del menú principal

El programa presenta un menú principal con trece opciones que engloban todos los subsistemas: gestionar personas de la liga, gestionar equipos, gestionar fichajes y plantillas, gestionar calendario, gestionar cola de partidos, registrar partidos jugados, gestionar incidencias y sanciones, mostrar clasificación, mostrar estadísticas, mostrar historial de acciones, deshacer última acción, gestionar apuestas y salir. Cada una de estas opciones deriva en submenús específicos que permiten realizar todas las operaciones detalladas.

2. EXPLICACIÓN DEL DISEÑO DE CLASES

2.1. Filosofía general del diseño orientado a objetos

El diseño del proyecto sigue los principios fundamentales de la programación orientada a objetos, incluyendo encapsulación, herencia, polimorfismo y abstracción. Todas las clases tienen sus atributos declarados como privados y proporcionan métodos getter y setter públicos para acceder y modificar dichos atributos de forma controlada. La jerarquía de clases permite reutilizar código y establecer relaciones lógicas entre los distintos elementos del dominio del problema.

2.2. Clase abstracta PersonaLiga

En primer lugar, se ha definido una clase abstracta llamada PersonaLiga, que actúa como superclase de Jugador y Entrenador. Esta clase contiene los atributos comunes a todas las personas que forman parte de la liga, como identificador, nombre, nickname, edad y salario base. Incluye métodos concretos como mostrarResumen, que muestra la información común de cualquier persona, y getters y setters para todos sus atributos. Además, declara un método abstracto llamado calcularCosteMensual, que debe ser implementado de forma distinta por

cada subclase. La decisión de hacer esta clase abstracta responde a la necesidad de tratar a todos los miembros de la liga de forma polimórfica, permitiendo almacenar objetos de tipo Jugador y Entrenador en una misma colección, mientras que el método abstracto fuerza a cada subclase a definir su propia lógica de cálculo salarial.

2.3. Interfaz Entrenable

Se ha definido una interfaz llamada Entrenable que declara dos métodos: entrenar y calcularRendimiento. Esta interfaz es implementada por la clase Jugador, aunque su diseño permite que en el futuro otras clases, como podrían ser los entrenadores o cualquier otro rol entrenable, también puedan implementarla. El uso de una interfaz en lugar de una clase abstracta permite mayor flexibilidad, ya que Java no soporta herencia múltiple pero sí implementación de múltiples interfaces.

2.4. Clase Jugador

La clase Jugador extiende PersonaLiga e implementa la interfaz Entrenable. Sus atributos específicos incluyen el rol, que es un enumerado con los valores TOP, JUNGLE, MID, ADC y SUPPORT, así como el nivel mecánico, el nivel estratégico, las partidas jugadas, los MVP totales y un booleano que indica si está sancionado o no. La clase implementa el método calcularCosteMensual de forma que el salario base se incrementa con cien euros por cada MVP total y se añaden trescientos euros adicionales si el jugador ha disputado quinientas o más partidas. El método mostrarResumen se sobrescribe para añadir la información específica del jugador. Por otra parte, al implementar la interfaz Entrenable, la clase Jugador proporciona el método entrenar, que incrementa en uno los niveles mecánico y estratégico, y el método calcularRendimiento, que devuelve un valor entre cero y diez calculado como la media de los niveles mecánico y estratégico, con un bonus de cinco puntos si la proporción de MVP respecto a partidas jugadas es igual o superior a 0.5, limitando el resultado a un máximo de diez.

2.5. Clase Entrenador

La clase Entrenador también extiende PersonaLiga e incorpora atributos como la experiencia en años, la especialidad y las victorias totales a lo largo de su carrera. Su método calcularCosteMensual se define como el salario base más diez euros por cada victoria total más cien euros por cada año de experiencia. El método mostrarResumen se sobrescribe para incluir estos datos específicos. Esta diferenciación en el cálculo del coste mensual entre jugadores y entrenadores ejemplifica claramente el uso del polimorfismo, ya que el sistema puede tratar a ambos tipos de forma uniforme mientras cada uno aplica su propia lógica de negocio.

2.6. Clase Equipo

La clase Equipo es una de las más relevantes del sistema. Contiene atributos como nombre, ciudad, entrenador, presupuesto, número de victorias, número de derrotas, puntos a favor y puntos en contra. Para la gestión de la plantilla, dispone de un array fijo de cinco posiciones para los jugadores titulares, reflejando la estructura inmutable de un equipo de e-sports donde

siempre hay cinco roles específicos, y una lista dinámica de tipo ArrayList para los jugadores suplentes, cuyo número puede variar a lo largo de la temporada.

2.6.1. Métodos de gestión de titulares

Los métodos más importantes de esta clase incluyen añadirTitular, que verifica que no exista ya un titular con el mismo rol y que el jugador no esté sancionado, lanzando una excepción TitularInvalidoException en caso contrario.

2.6.2. Métodos de gestión de suplentes

El método añadirSuplente comprueba que el jugador no sea ya titular y que no esté ya en la lista de suplentes, lanzando SuplenteInvalidoException si se incumple alguna de estas condiciones.

2.6.3. Método de promoción de suplentes

El método promoverSuplente permite mover un jugador del banquillo al once titular en una posición determinada, eliminándolo de la lista de suplentes y asignándolo al array de titulares en la posición indicada, con las correspondientes validaciones.

2.6.4. Método de cálculo de costes

Finalmente, el método calcularCosteTotal recorre todos los titulares y suplentes sumando sus costes mensuales y añade el coste mensual del entrenador.

2.7. Clase Partido

La clase Partido almacena toda la información relativa a un enfrentamiento, incluyendo un identificador único, la jornada en la que se disputa, el equipo local, el equipo visitante, la puntuación del local, la puntuación del visitante, el jugador MVP y un estado booleano que indica si el partido ya ha sido disputado o no.

2.7.1. Método registrarResultado

El método registrarResultado es el más relevante de esta clase. En primer lugar, verifica que el partido no haya sido disputado ya, lanzando PartidoDisputadoException en caso contrario, y que los dos equipos sean diferentes, lanzando PartidoInvalidoException si son el mismo. A continuación, comprueba que el jugador designado como MVP pertenezca realmente a alguno de los dos equipos recorriendo los arrays de titulares de ambos, y si no es así lanza MvpInvalidoException.

2.7.2. Actualización de estadísticas

Una vez superadas todas las validaciones, actualiza las puntuaciones, marca el partido como disputado, calcula el ganador mediante el método calcularGanador, y actualiza las estadísticas de los equipos incrementando victorias, derrotas, puntos a favor y puntos en contra según

corresponda. Además, incrementa en uno las partidas jugadas de todos los jugadores titulares de ambos equipos y suma un MVP al jugador designado.

2.8. Clase Incidencia

La clase Incidencia representa cualquier evento anómalo que pueda ocurrir durante la temporada. Sus atributos son un identificador, una descripción textual, un tipo que es un enumerado con valores como SANCION, EXPULSION, ERROR_TECNICO y PARTIDO_APLAZADO, y referencias al equipo y al jugador implicados.

2.8.1. Método aplicarSancion

El método aplicarSancion verifica si el tipo de incidencia es SANCION o EXPULSION y, en ese caso, marca al jugador como sancionado, lanzando una excepción JugadorInvalidoException si el jugador es nulo.

2.9. Clase Apuesta

La clase Apuesta es la funcionalidad extra del proyecto. Permite a los usuarios apostar por un equipo en un partido pendiente. Sus atributos son el equipo apostado, la cantidad apostada en euros, la cuota calculada dinámicamente, el partido asociado y dos booleanos que indican si la apuesta ya ha sido resuelta y si ha sido ganada.

2.9.1. Cálculo de la cuota

La cuota se calcula mediante una fórmula que relaciona victorias y derrotas, concretamente uno más el resultado de dividir las derrotas más uno entre las victorias más uno, de manera que los equipos con peor historial ofrecen cuotas más altas al ser considerados menos probables ganadores.

2.10. Clase Liga

La clase Liga es la clase contenedora principal del sistema y actúa como núcleo de la lógica de negocio. Agrupa todas las colecciones necesarias para el funcionamiento de la aplicación, incluyendo un ArrayList de equipos, otro de personas de la liga, otro de incidencias, una cola de tipo Queue para los partidos pendientes implementada como LinkedList, una pila de tipo Stack para el historial de acciones, un HashSet para los identificadores ya registrados, otro HashSet para los nombres de equipos ya existentes, una matriz bidimensional para el calendario, un ArrayList para las apuestas y un double que representa el saldo del administrador.

2.10.1. Gestión de personas y equipos

Entre sus métodos más importantes se encuentran registrarPersona, que comprueba si el identificador ya existe en el HashSet y lanza PersonaDuplicadaException en caso afirmativo, registrarEquipo con una lógica similar usando el HashSet de nombres, listarPersonas,

listarEquipos, buscarPersona y buscarEquipo que recorren las colecciones correspondientes, y eliminarPersona y eliminarEquipo que eliminan elementos y actualizan los HashSet de control.

2.10.2. Gestión de la cola de partidos

En cuanto a la cola de partidos, los métodos encolarPartido, mostrarSiguientePartido, mostrarColaPartidos y disputarSiguientePartido gestionan el comportamiento FIFO, siendo este último el que extrae el partido de la cola mediante poll, registra su resultado y resuelve las apuestas asociadas.

2.10.3. Gestión del historial

Para el historial, los métodos registrarAccion, mostrarUltimaAccion, mostrarHistorial y deshacerUltimaAccion gestionan el comportamiento LIFO de la pila.

2.10.4. Gestión de incidencias y clasificación

La clase Liga también incluye métodos para gestionar incidencias, como registrarIncidencia, listarIncidencias y las búsquedas por equipo o jugador, así como mostrarClasificacion, que ordena dinámicamente la lista de equipos con un Comparator que prioriza victorias, luego diferencia de puntos y finalmente nombre del equipo.

2.10.5. Gestión del calendario

El calendario se gestiona con generarCalendario, que crea una matriz de equipos por jornadas inicializada a cero, mostrarCalendario y mostrarJornada.

2.10.6. Gestión de apuestas

Finalmente, el sistema de apuestas se integra mediante realizarApuesta, que calcula la cuota, resta la cantidad del saldo del administrador y añade la apuesta a la lista, y resolverApuestas, que se ejecuta automáticamente al disputar un partido, recorre todas las apuestas asociadas a ese partido y, si el equipo apostado resultó ganador, incrementa el saldo del administrador con la cantidad multiplicada por la cuota.

2.11. Enumerados

Se han definido dos enumerados en el proyecto. El primero es Rol, con los valores TOP, JUNGLE, MID, ADC y SUPPORT, que define las posiciones que pueden ocupar los jugadores en el campo. El segundo es TipoIncidencia, con los valores SANCION, EXPULSION, ERROR_TECNICO y PARTIDO_APLAZADO, que clasifica los diferentes tipos de incidencias que pueden registrarse.

2.12. Clase Main

La clase Main contiene el método main, el bucle principal del menú y todos los métodos auxiliares necesarios para la interacción por consola. Se han implementado métodos como

pedirString, pedirInt, pedirIntRango, pedirDouble, pedirDoubleRango y pedirBoolean para manejar de forma robusta la entrada de datos por teclado, con validaciones de rango y control de errores. Cada opción del menú principal deriva en un submenú específico cuyas opciones llaman a métodos estáticos que solicitan los datos necesarios al usuario e invocan los métodos correspondientes de la clase Liga. Además, el método inicializarDatos crea y registra una serie de objetos de ejemplo para que el programa pueda ser probado sin necesidad de introducir todos los datos desde cero, incluyendo seis entrenadores, treinta jugadores titulares, seis jugadores suplentes, seis equipos con sus respectivas plantillas completas y cinco partidos encolados.

3. JUSTIFICACIÓN DEL USO DE ESTRUCTURAS DINÁMICAS

3.1. Criterios generales de selección

El proyecto hace un uso extensivo y justificado de diferentes estructuras de datos, cada una elegida por su comportamiento específico y por su adecuación al problema que resuelve. A continuación se explica detalladamente por qué se ha seleccionado cada estructura y qué ventaja aporta al sistema.

3.2. Arrays fijos para titulares de equipo

Para los jugadores titulares de cada equipo se ha utilizado un array fijo de cinco posiciones. Esta decisión se debe a que un equipo de e-sports tiene exactamente cinco roles fijos en el campo de juego, que son TOP, JUNGLE, MID, ADC y SUPPORT. Este número es inmutable a lo largo de toda la temporada, por lo que un array de tamaño fijo refleja perfectamente esta realidad. Además, el array permite un acceso directo por índice, lo que resulta muy útil cuando se necesita acceder a la posición de un rol concreto o cuando se promueve a un suplente a una posición específica. El uso de un array también comunica claramente a cualquier persona que lea el código que el número de titulares es fijo e inamovible, lo que mejora la legibilidad y la intencionalidad del diseño.

3.3. ArrayList para suplentes y colecciones principales

Para los jugadores suplentes, en cambio, se ha utilizado una lista dinámica de tipo ArrayList. El número de suplentes puede variar a lo largo de la temporada, ya que un equipo puede fichar nuevos jugadores, despedir a otros, o ascender canteranos. Una lista dinámica es la estructura adecuada cuando el tamaño de la colección puede crecer o decrecer de forma impredecible. El ArrayList ofrece métodos como add para añadir nuevos suplentes, remove para eliminarlos y un iterador para recorrerlos, todas ellas operaciones necesarias en el sistema. Además, el ArrayList mantiene el orden de inserción, lo que es útil para mostrar la lista de suplentes en el orden en que fueron incorporados al equipo. Fuera de la clase Equipo, se han utilizado también

ArrayLists para las colecciones principales de la clase Liga, como la lista de equipos, la lista de personas, la lista de incidencias y la lista de apuestas. La razón es la misma que en el caso anterior: no sabemos de antemano cuántos equipos se van a registrar, cuántas personas van a formar parte de la liga ni cuántas incidencias o apuestas van a ocurrir. El ArrayList proporciona la flexibilidad necesaria para manejar colecciones de tamaño variable y además permite búsquedas, inserciones y eliminaciones de forma eficiente en la mayoría de los casos de uso de la aplicación.

3.4. Queue para la cola de partidos pendientes

Para los partidos pendientes se ha utilizado una cola, implementada mediante la interfaz Queue con una LinkedList subyacente. La cola es una estructura con comportamiento FIFO, es decir, el primer elemento que entra es el primero que sale. Este comportamiento es exactamente el que necesita un calendario de partidos, donde los partidos se programan en un orden determinado y deben disputarse en ese mismo orden. Los métodos offer para encolar un nuevo partido, poll para extraer el siguiente partido a disputar y peek para consultar cuál es el siguiente sin extraerlo reflejan a la perfección las operaciones que el sistema debe realizar sobre los partidos pendientes. El uso de una cola comunica de forma clara e inequívoca la política de ordenación que sigue el sistema.

3.5. Stack para el historial de acciones

Para el historial de acciones se ha utilizado una pila, implementada mediante la clase Stack. La pila es una estructura con comportamiento LIFO, es decir, el último elemento que entra es el primero que sale. Este comportamiento es el adecuado para implementar una funcionalidad de deshacer, ya que las acciones más recientes son las primeras que el usuario querrá deshacer. El método push registra una nueva acción en el historial, el método pop deshace la última acción extrayéndola de la pila, y el método peek permite consultar la última acción sin deshacerla. La pila refleja de forma natural la lógica de navegación hacia atrás en el historial de operaciones.

3.6. HashSet para el control de duplicados

Para el control de duplicados se han utilizado dos HashSet, uno para los identificadores de personas ya registradas y otro para los nombres de equipos ya existentes. La principal ventaja del HashSet es que ofrece un tiempo de búsqueda constante, es decir $O(1)$, para verificar si un elemento ya existe en el conjunto. Esto es crucial porque las operaciones de registro de personas y equipos se realizan con frecuencia, y una búsqueda lineal en una lista sería ineficiente a medida que el número de elementos crece. El HashSet también garantiza que no haya elementos duplicados por definición, aunque en este caso se utiliza como mecanismo de comprobación previa antes de insertar en las listas principales.

3.7. Matriz bidimensional para el calendario

Para el calendario se ha utilizado una matriz bidimensional de enteros. La matriz relaciona dos dimensiones: los equipos en las filas y las jornadas en las columnas. Cada celda almacena los

puntos que ese equipo ha obtenido en esa jornada, típicamente tres por victoria, uno por empate y cero por derrota. La matriz es la representación más natural y eficiente para este tipo de dato tabular. Permite consultar rápidamente cuántos puntos tiene un equipo en una jornada concreta mediante un acceso directo por índices, y también permite recorrer fácilmente todas las jornadas de un equipo o todos los equipos en una jornada. Una estructura unidimensional no sería adecuada porque no capturaría la relación bidimensional entre equipos y jornadas, mientras que una lista de listas sería menos eficiente en el acceso y más compleja de manejar.

3.8. Iterator para recorridos seguros

Finalmente, se ha utilizado Iterator de forma sistemática para recorrer las colecciones, especialmente en aquellos casos donde es necesario eliminar elementos durante la iteración, como ocurre en los métodos eliminarPersona y eliminarEquipo. El Iterator proporciona un método remove seguro que evita las excepciones de modificación concurrente que aparecerían si se intentara eliminar un elemento directamente mientras se recorre una colección con un bucle for-each.

4. EXPLICACIÓN DE LAS EXCEPCIONES

4.1. Filosofía general del manejo de excepciones

El proyecto implementa un control de errores robusto mediante el uso de excepciones, tanto estándar como personalizadas. Todas las excepciones personalizadas creadas extienden la clase Exception, lo que las convierte en excepciones comprobadas, es decir, el compilador obliga a tratarlas mediante un bloque try-catch o a declararlas en la cláusula throws del método que puede lanzarlas. Esta decisión de diseño es intencionada, ya que los errores que representan estas excepciones son situaciones previsibles y recuperables, como intentar registrar una persona con un identificador duplicado o disputar un partido que ya ha sido jugado, y el programa debe estar preparado para manejarlas de forma adecuada informando al usuario y permitiéndole corregir su acción.

4.2. Excepciones relacionadas con personas

En total se han creado once excepciones personalizadas. La primera de ellas es PersonaDuplicadaException, que se lanza en el método registrarPersona de la clase Liga cuando el identificador de la persona que se intenta registrar ya existe en el HashSet idsRegistrados. Esta excepción evita que dos personas distintas compartan el mismo identificador, lo que sería un error grave de integridad de los datos. Por su parte, PersonaNoEncontradaException se lanza en el método eliminarPersona cuando no se encuentra ninguna persona con el identificador proporcionado, así como en otros métodos que buscan una persona existente.

4.3. Excepciones relacionadas con equipos

En el ámbito de los equipos, `EquipoDuplicadoException` se lanza al intentar registrar un equipo cuyo nombre ya está presente en el `HashSet` `nombresEquipos`, garantizando la unicidad de los nombres de equipo en la liga. `EquipoNoEncontradoException` se lanza en `eliminarEquipo` cuando no se encuentra un equipo con el nombre especificado.

4.4. Excepciones relacionadas con plantillas

Las excepciones relacionadas con la gestión de plantillas son `TitularInvalidoException` y `SuplenteInvalidoException`. La primera se lanza en el método `añadirTitular` de la clase `Equipo` cuando se intenta añadir un jugador como titular que tiene un rol que ya está cubierto por otro titular, o cuando el jugador está sancionado. También se lanza en `promoverSuplente` si la posición indicada está fuera del rango válido o ya está ocupada. `SuplenteInvalidoException` se lanza en `añadirSuplente` si el jugador ya es titular o ya está en la lista de suplentes, en `eliminarSuplente` si el jugador no se encuentra entre los suplentes, y en `promoverSuplente` si el jugador no está en la lista de suplentes.

4.5. Excepciones relacionadas con partidos

En cuanto a los partidos, se han definido tres excepciones. `PartidoDisputadoException` se lanza tanto en `encolarPartido` de la clase `Liga`, cuando se intenta encolar un partido que ya ha sido disputado, como en `registrarResultado` de la clase `Partido`, cuando se intenta registrar el resultado de un partido que ya está marcado como disputado. `PartidoInvalidoException` se lanza en las mismas circunstancias cuando el equipo local y el equipo visitante son el mismo equipo, lo que no tiene sentido en una competición real. `MvpInvalidoException` se lanza en `registrarResultado` cuando el jugador designado como MVP no pertenece a ninguno de los dos equipos que disputan el partido, lo que se comprueba recorriendo los arrays de titulares de ambos equipos.

4.6. Excepciones relacionadas con jugadores y apuestas

`JugadorInvalidoException` se lanza en el método `aplicarSancion` de la clase `Incidencia` cuando se intenta aplicar una sanción o expulsión a un jugador que es nulo, es decir, cuando la incidencia no tiene asociado ningún jugador a pesar de ser de un tipo que debería tenerlo. Por último, `SaldoInsuficienteException` se lanza en el método `realizarApuesta` de la clase `Liga` cuando la cantidad que el usuario intenta apostar es superior al saldo disponible del administrador, evitando que se realicen apuestas por importes superiores a los fondos reales.

4.7. Manejo de excepciones estándar

Además de estas excepciones personalizadas, el sistema también maneja situaciones que podrían provocar excepciones estándar de Java, como `NullPointerException`, mediante comprobaciones explícitas de nulidad antes de acceder a métodos de objetos potencialmente nulos. Por ejemplo, en los métodos `buscarPersona` y `buscarEquipo` se verifica que el objeto encontrado no sea nulo antes de retornarlo, y en los métodos que reciben objetos como parámetros se realizan comprobaciones similares.

4.8. Separación de capas en el manejo de excepciones

La filosofía de diseño de excepciones seguida en el proyecto establece una clara separación entre la capa de lógica de negocio y la capa de presentación. Los métodos de las clases Liga, Equipo, Partido e Incidencia son los que lanzan las excepciones cuando detectan una situación irregular, pero no se encargan de capturarlas ni de mostrar mensajes al usuario. En cambio, los métodos de la clase Main que interactúan directamente con el usuario son los que contienen los bloques try-catch. En estos bloques, se captura la excepción, se extrae su mensaje mediante getMessage y se muestra al usuario de forma amigable, permitiéndole entender qué ha ocurrido y cómo puede corregir su acción. Esta separación de responsabilidades mejora la mantenibilidad del código, ya que la lógica de negocio no depende de la interfaz de usuario concreta que se utilice.

4.9. Mensajes de error descriptivos

Cada excepción personalizada incluye un constructor que recibe un mensaje descriptivo como parámetro y lo pasa a la superclase Exception mediante super mensaje. De esta forma, cuando se lanza la excepción, el mensaje describe con precisión cuál ha sido el problema, como por ejemplo "No pueden haber dos titulares con el mismo rol" o "El partido ya se ha disputado". Estos mensajes se muestran directamente al usuario, proporcionando una experiencia informativa que le ayuda a corregir su entrada sin necesidad de buscar documentación adicional.

5. CONCLUSIÓN

5.1 Programación estructurada y modular

En cuanto a la programación estructurada y modular, el código está organizado en clases con responsabilidades bien definidas y una separación clara entre la lógica de negocio y la interfaz de usuario. Los métodos son concisos y realizan una única tarea, lo que facilita su comprensión y mantenimiento. El menú principal y los submenús están estructurados de forma jerárquica y todas las funcionalidades son accesibles a través de opciones numeradas.

5.3. Programación orientada a objetos

Respecto a la programación orientada a objetos, se han utilizado todos sus principios fundamentales. La encapsulación está presente mediante atributos privados y métodos públicos getter y setter. La herencia se manifiesta en la jerarquía que tiene a PersonaLiga como clase abstracta y a Jugador y Entrenador como subclases. El polimorfismo se ejemplifica en el método calcularCosteMensual, que se comporta de forma diferente según la subclase que lo ejecute, y en el tratamiento uniforme de jugadores y entrenadores como objetos de tipo

PersonaLiga dentro de las colecciones. La abstracción está representada por la clase PersonaLiga, que declara métodos abstractos sin implementar, y por la interfaz Entrenable, que define un contrato que deben cumplir las clases que la implementan.

5.4. Arrays, matrices y estructuras dinámicas

En cuanto a arrays y matrices, se ha utilizado un array fijo de cinco posiciones para los titulares de cada equipo, reflejando la estructura inmutable de un equipo de e-sports, y una matriz bidimensional para el calendario, que relaciona equipos y jornadas de forma natural y eficiente. Las estructuras dinámicas se han seleccionado cuidadosamente según su comportamiento específico. Los ArrayList se utilizan para colecciones de tamaño variable como suplentes, equipos, personas, incidencias y apuestas. La Queue con comportamiento FIFO es la estructura ideal para la cola de partidos pendientes. El Stack con comportamiento LIFO es perfecto para el historial de acciones con capacidad de deshacer. Los HashSet proporcionan búsqueda en tiempo constante $O(1)$ para el control de duplicados. Y los Iterator permiten recorridos seguros con eliminación durante la iteración.

5.5. Excepciones personalizadas

El control de errores mediante excepciones es exhaustivo, con once excepciones personalizadas que cubren todas las situaciones irregulares previsibles, desde duplicados y ausencias hasta validaciones de negocio como roles repetidos, jugadores sancionados o partidos inválidos. Cada excepción se lanza en la capa de lógica de negocio y se captura en la capa de presentación, con mensajes claros que informan al usuario.

5.6. Interfaz Entrenable

La interfaz Entrenable está implementada por la clase Jugador, que proporciona los métodos entrenar y calcularRendimiento, este último con una lógica que combina niveles mecánico y estratégico con la proporción de MVP por partida.

5.7. Funcionalidad extra de apuestas

Finalmente, el sistema de apuestas añade un valor significativo al proyecto. Permite a los usuarios apostar cantidades de dinero ficticio sobre equipos en partidos pendientes, con cuotas calculadas dinámicamente mediante una fórmula basada en las victorias y derrotas del equipo en el momento de la apuesta, que es uno más el resultado de dividir las derrotas más uno entre las victorias más uno. Las apuestas se resuelven automáticamente al disputarse el partido, incrementando el saldo del administrador si la apuesta fue ganada, y todo el historial de apuestas queda registrado para su consulta posterior.

5.8. Valoración final

En conclusión, la aplicación es completamente funcional, robusta frente a entradas incorrectas del usuario, fácilmente ampliable para futuros requisitos, y demuestra de forma fehaciente el dominio de todos los contenidos trabajados durante el curso, incluyendo programación

estructurada, programación modular, programación orientada a objetos, arrays, matrices, excepciones, clases abstractas, interfaces y estructuras dinámicas. El sistema de apuestas aporta un valor añadido que va más allá de lo exigido en el enunciado y que demuestra una comprensión profunda del diseño de software y la capacidad de integrar funcionalidades complejas de forma coherente con el resto del sistema.